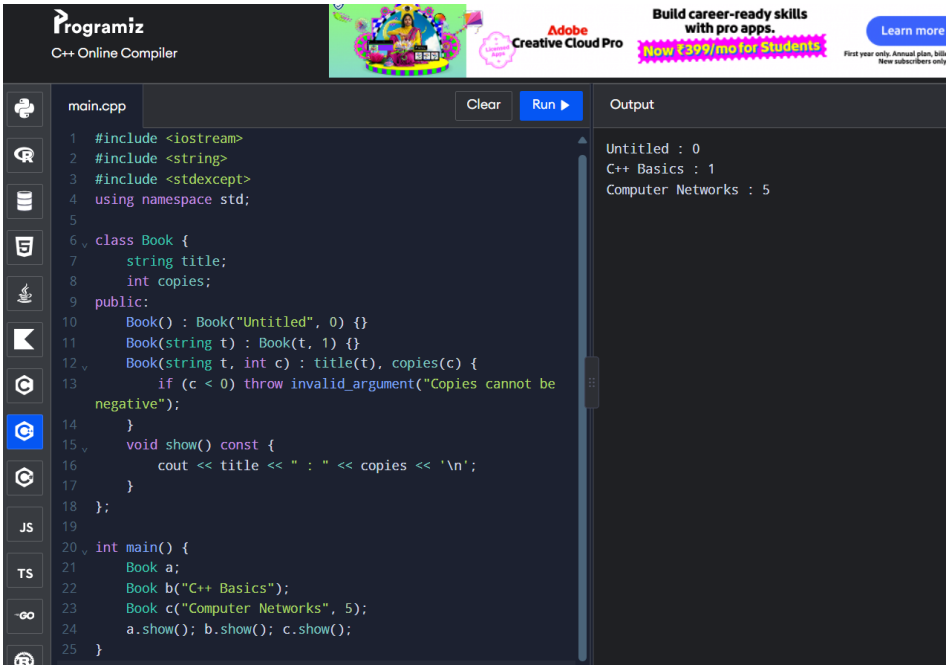
ASSIGNMENT 3 – CO3

Constructors, Destructors and Operator Overloading

Q1 – Constructors and Constructor Overloading

Program



Answers / Design and Analysis

- A default constructor is supplied by the compiler only when the class has no user-declared constructor. If any constructor is declared, the compiler does not automatically add a no-argument default constructor.
- A member-initializer list initializes data members directly before the constructor body runs. Assignment in the body first default-constructs the member and then assigns a value. Direct initialization is necessary for const members and reference members and is generally more efficient.
- Members are initialized in the order in which they are declared in the class, not the order written in the initializer list.
- Object states: a = (title="Untitled", copies=0); b = ("C++ Basics", 1); c = ("Computer Networks", 5). Negative copies throw invalid_argument, so an invalid object is not successfully constructed.
- Constructor delegation is used by the default and title-only constructors. They call the two-argument constructor, keeping validation in one place.

- Use explicit for a constructor that accepts one argument when implicit conversion is not intended. Without explicit, Book b = "Novel"; may compile if a matching converting constructor exists. With explicit, copy-initialization is rejected; direct initialization Book b("Novel"); still works.

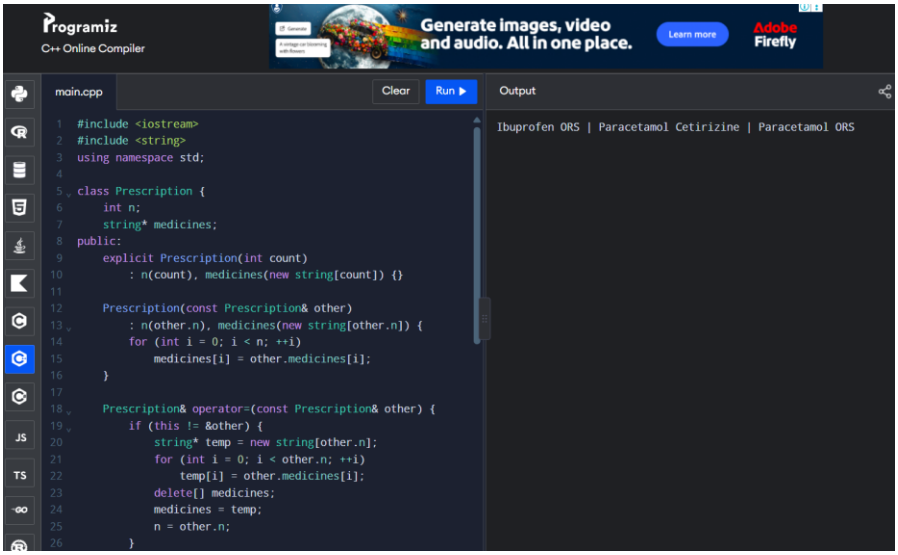
If a single constructor Book(string t="", int c=0) were used, all creation cases could be handled by default arguments. Both designs create one object in O(1) time and O(1) extra space. Overloaded constructors with delegation make the intended cases clearer. Centralizing validation in the two-argument constructor checks copies in one place.

Experimental checks

- Create the three objects and print them. Try Book d("Invalid", -2); inside try/catch; the exception message should be displayed.
- If the default constructor is removed, Book a[3]; fails because the array requires a usable default constructor for each element.
- To demonstrate -Wreorder, deliberately list initializers in a different order from member declarations, then compile with -Wreorder. The compiler warns that actual initialization still follows declaration order.

Q2 – Copy Constructors: Shallow and Deep Copies (7 Marks)

Completed Code



Answers / Design and Analysis

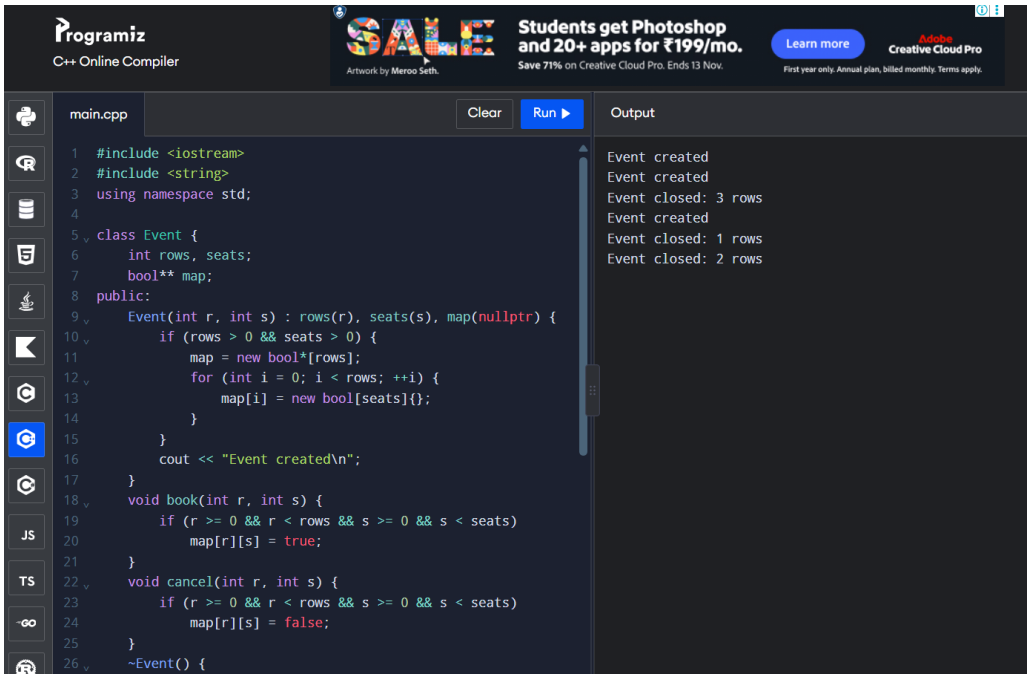
- Shallow copy copies the pointer address. Both objects then refer to the same medicine array. Changing one object changes the other, and both destructors try to delete the same memory, causing a double-free error.
- Deep copy allocates a separate array and copies every medicine string. The original and copied prescriptions are independent.
- Rule of Three: when a class manages a resource requiring a destructor, it commonly needs a copy constructor and copy-assignment operator as well.
- The copy constructor is called when an object is initialized from another object, when an object is passed by value, and when an object is returned by value (subject to copy elision).
- Passing by value creates a copy; passing by const reference does not create a copy. The copy constructor message can be used to count copies.
- Expected output: Ibuprofen ORS | Paracetamol Cetirizine | Paracetamol ORS.
- Self-assignment p = p is safe because the operator checks this != &other before releasing memory.

For n medicines, deep copy takes O(n) time and O(n) additional space. A shallow pointer copy is O(1), but it is unsafe when both objects believe they own the same memory. With vector<string>, the compiler-generated copy is already a deep value copy; this is the Rule of Zero and normally removes the need to write the three special member functions.

Sanitizer: compile with g++ -std=c++17 -fsanitize=address -g file.cpp -o app. The correct version should finish without memory errors. The shallow version may report attempting double-free or heap-use-after-free. Exact diagnostic wording depends on the sanitizer version.

Q3 – Dynamic Constructors and Destructors (7 Marks)

Program



Answers / Design and Analysis

- A local object is constructed when execution reaches its declaration and destroyed when its scope ends. A block-scope object is destroyed at the closing brace. A dynamically allocated object is destroyed only when delete is called.
- For arrays allocated with new[], use delete[]. Using delete instead of delete[] is undefined behaviour. Forgetting delete causes a memory leak.
- A destructor has no parameters and no return type. It cannot be overloaded because a class has only one destructor signature.
- Memory layout: map points to an array of row pointers; each row pointer points to a separately allocated array of bool values. The destructor deletes each row, then deletes the row-pointer array.
- Zero rows: map remains nullptr; the destructor safely skips deletion.

RAII ties resource release to object lifetime. When the object leaves scope, its destructor automatically releases owned memory, reducing leaks and making cleanup predictable.

For a rows × seats map, bool storage is approximately rows × seats bytes (implementation-dependent), plus rows pointers and allocation overhead. A 10×20 map stores 200 bool values; 100×200 stores 20,000; 1,000×2,000 stores 2,000,000. Report measured bytes from your compiler if required.

Predict the Output – Logger

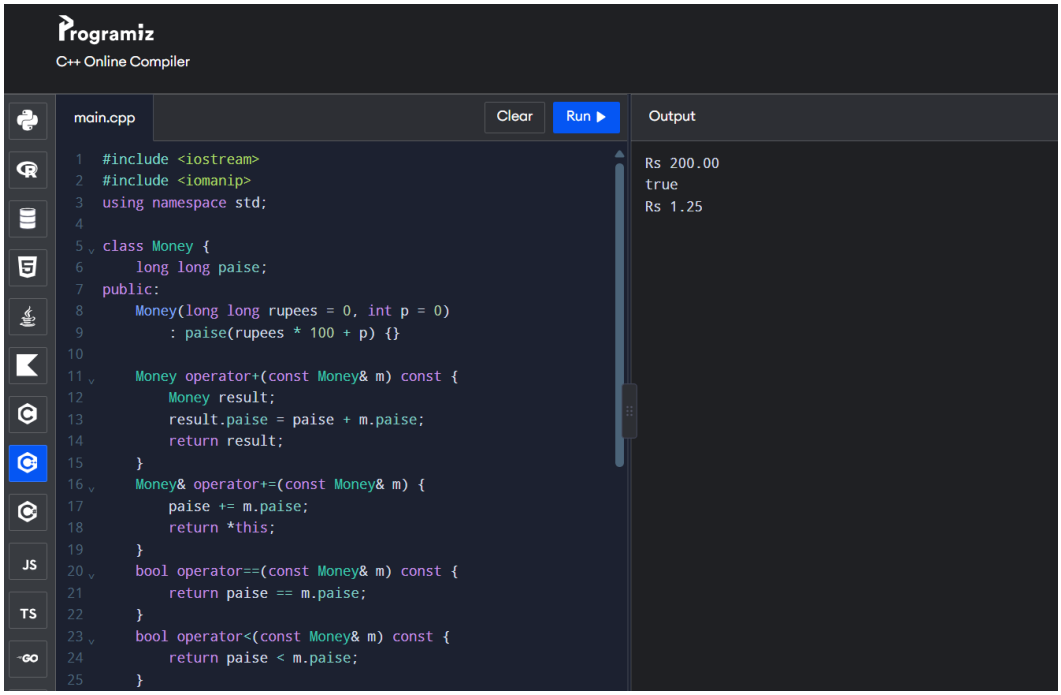
+A +B -B +A copyAc use -Ac +P -P end -A

The exact spacing follows the stream statements. A and B are constructed in order. B is destroyed at the end of its inner block. use(a) passes a by value, so a copy named Ac is constructed; the parameter is destroyed when use returns. P is dynamically constructed and destroyed by delete. Finally, end is printed and a is destroyed at the end of main.

A contiguous rows×seats block needs one allocation and uses index r*seats+s. A vector<vector<char>> manages row storage automatically. Both provide O(1) indexed access. A row-pointer design uses rows+1 allocations and extra row-pointer storage; one contiguous block uses one allocation.

Q4 – Overloading Arithmetic and Stream Operators (7 Marks)

Completed Code



Answers / Design and Analysis

- + returns a new Money object and leaves its operands unchanged. += changes the current object and returns Money& (*this), allowing chained assignments such as a += b += c.
- Arithmetic and comparison operators can be members because the left operand is a Money object. Stream operators are non-member friends because the left operand of cout << money is ostream, not Money.

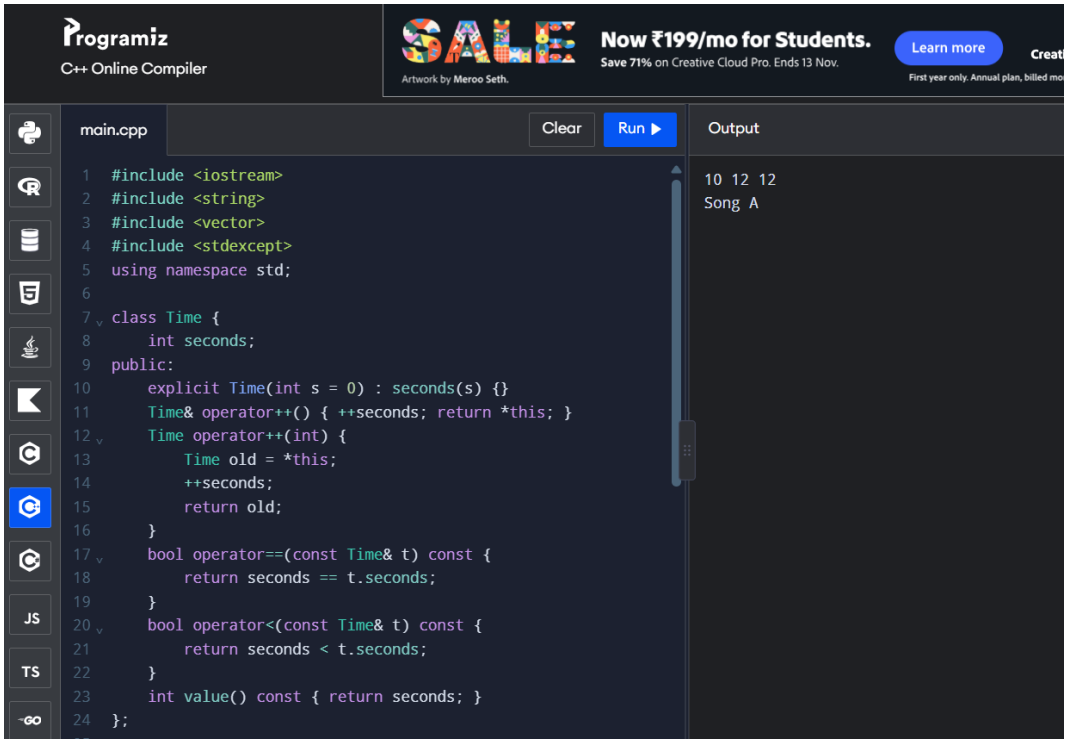
- a + b is treated as a.operator+(b) for this member implementation.
- Expected main output: Rs 200.00, true, and Rs 1.25 for the paise-carry test.
- A negative subtraction result can be represented by signed paise. The application should decide whether negative balances are permitted; otherwise reject a result below zero.

Member and friend implementations both take O(1) time and O(1) extra space for these operations. A non-member friend operator can allow conversions on either operand, such as 100 + m, if a suitable converting constructor exists. Named functions add() and equals() are explicit but less natural to read.

Multiplication of one Money value by a numeric quantity makes sense (for example, price × quantity). Multiplying one money amount by another money amount generally has no useful billing meaning.

Q5 – Overloading Increment, Comparison and Subscript Operators (6 Marks)

Program



Answers / Design and Analysis

- The compiler distinguishes prefix ++i from postfix i++ using the dummy int parameter in operator++(int). Prefix has no dummy parameter.

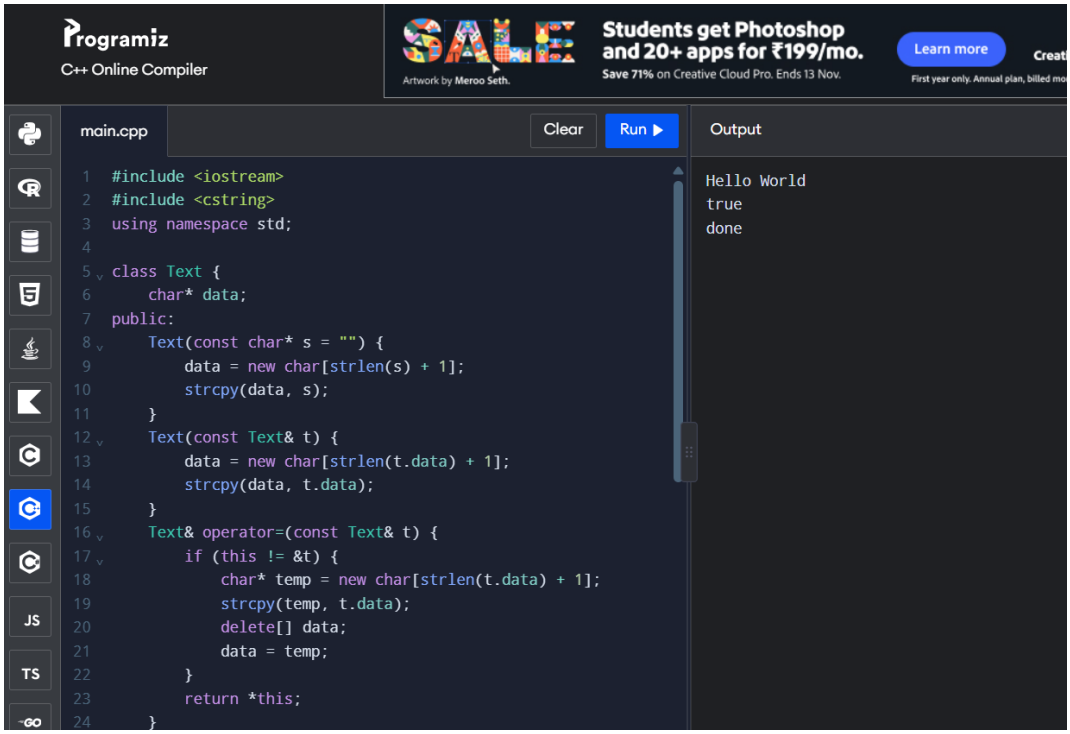
- Prefix increments the object and returns a reference to the updated object. Postfix saves the old value, increments the object and returns the saved copy, so postfix may require an extra copy.
- A non-const operator[] returns string& so the caller can modify a track. The const version returns const string& and allows read-only access on const playlists.
- An out-of-range index should be handled safely, for example by throwing out_of_range. A negative index should be rejected before converting to an unsigned index.
- For t=10: a=t++ stores 10 and t becomes 11; b=++t stores 12 and t remains 12. Output: 10 12 12.

Prefix is often preferred for class types when the old value is not needed because it avoids creating and returning the extra old-value copy. Both forms are O(1) for this Time implementation. A checked subscript has one extra comparison and remains O(1); at(i) can throw out_of_range, while an unchecked subscript has no range check.

Minute/hour rollover should be implemented by storing total seconds and formatting it as minutes and seconds, or by explicitly carrying 60 seconds into a minute and 60 minutes into an hour.

Q6 – The Rules of Operator Overloading (7 Marks)

Program



Answers / Design and Analysis

- Operators that cannot be overloaded: :: (scope resolution), . (member access), .* (pointer-to-member access), ?: (conditional), and sizeof. Their meaning is part of the language syntax and cannot be changed by a class.
- Overloading cannot change an operator's precedence, associativity or number of operands (arity). At least one operand must be a user-defined type.
- operator= must be a member function because the language requires assignment overloads to be members of the class being assigned to.
- A friend is useful when an operator needs access to private members but the left operand is not the class type, such as ostream << Text.
- In a + b + c, the intermediate result of a+b may be copied or moved into the next operation; compiler move optimizations and copy elision can affect the observed count. Instrument the copy constructor to measure actual copies.
- Self-assignment is handled by checking this != &t. The program should also be tested with empty strings and long strings under AddressSanitizer.

For strings of lengths n1, n2 and n3, left-associated (a+b)+c copies approximately n1+n2 characters for the first result and then n1+n2+n3 for the next result: O(n1+n2+n3) total character work for two concatenations, with intermediate storage. Repeated concatenation can cause additional allocations. std::string is preferred in normal applications because it already manages memory safely. A custom Text class is useful for learning ownership or implementing specialized storage.

The alternative operator+(Text a, const Text& b) { a += b; return a; } can reuse the by-value left operand, reducing an extra intermediate allocation in suitable implementations. std::string provides optimized and well-tested memory management.

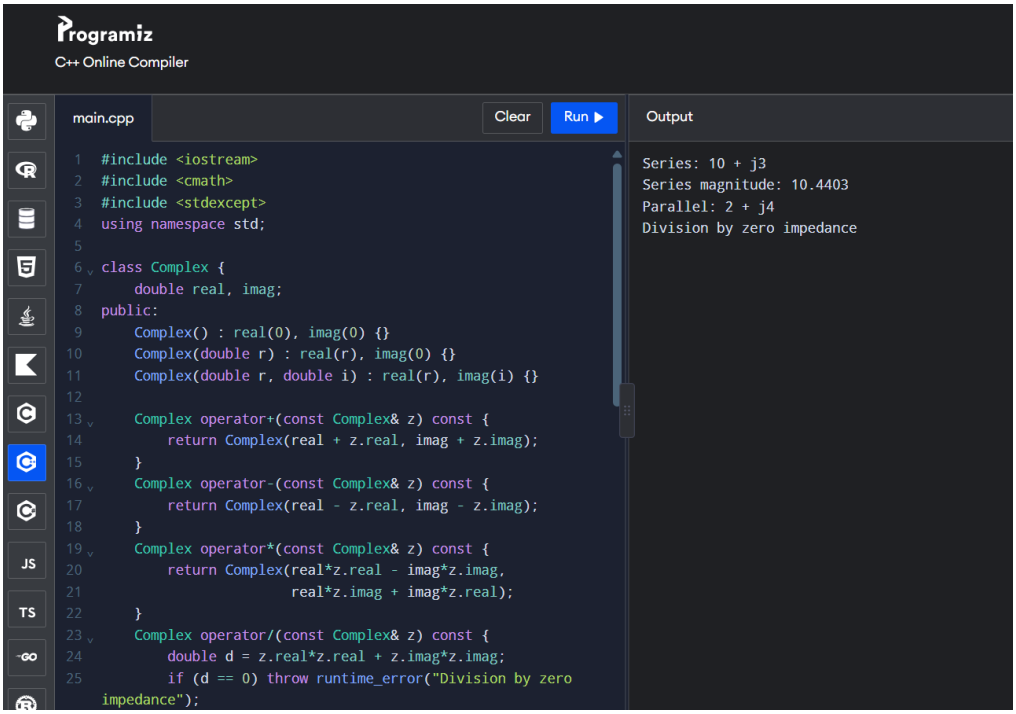
Q7 – Design a Complex-Number Class from First Principles (10 Marks)

Requirements and Formulae

- Data: real part (a) and imaginary part (b), both stored as double. Operations: addition, subtraction, multiplication, division, equality, printing and magnitude.
- Constructors: default Complex() gives 0+0j; Complex(double r) gives r+0j; Complex(double r, double i) gives r+ij.
- Addition: (a+bi)+(c+di) = (a+c)+(b+d)i.
- Subtraction: (a+bi)-(c+di) = (a-c)+(b-d)i.
- Multiplication: (a+bi)(c+di) = (ac-bd)+(ad+bc)i.

- Division: ((a+bi)/(c+di)) = ((ac+bd)/(c²+d²))+((bc-ad)/(c²+d²))i. If c²+d² is zero, report division by zero.
- Magnitude: |a+bi| = sqrt(a²+b²).

Implementation



Circuit Verification

For the sample values R=10 Ω, ZL=j5 Ω and ZC=−j2 Ω, the series impedance is 10+j3 Ω. Its magnitude is $\sqrt{(10^2+3^2)}=\sqrt{109}\approx 10.44$ Ω.

For R=10 and L=j5, the parallel impedance is (10×j5)/(10+j5). Multiplying numerator and denominator by (10−j5) gives (250+j500)/125 = 2+j4 Ω. The program's parallel example uses these two components.

Division by Complex(0,0) throws an exception and is caught in main, so the program prints the error message instead of dividing by zero.

Design Defence and Complexity

- The class is a small custom alternative to std::complex<double>. Both store two double values and perform arithmetic in O(1) time and O(1) space.
- The custom class makes the required operations and error handling visible and allows a chosen output format. std::complex<double> is standard, tested and usually preferable in production code.
- Constructors support the required forms of impedance. +, -, *, / implement circuit arithmetic; == compares values; << prints rectangular form.

- One limitation is exact floating-point equality: rounding can make mathematically equal values compare unequal. A tolerance-based comparison is safer for measured values.

For a real RLC circuit, inductive reactance is $X_L=\omega L$ and capacitive reactance is $X_C=1/(\omega C)$. The supplied sample uses already calculated reactances as imaginary components.